Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Mowbray

Faculty: Information Technology

Module Code: ITOPA3-33

Group: 1

Lecturer's Name: Bongani Mahlangu

Student Full Name: Joshua Nehohwa

Student Number: EDUV4948467

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature	Date

jnehohwa	28 August 2026

Lecturer's Comments:

Marks Awarded: %

Signature	Date

Table of Contents

Operating Systems: System Calls, Multithreading, Deadlock and File Management

Question 1 - Operating-system system calls

1.1 Basic file-management system calls and their importance

When a customer requests a bank statement, the banking application should not communicate with the disk by itself. It should make controlled requests to the operating system through system calls. The main file-management calls and their relevance are summarised below. Eduvos explains that system calls let user programs request protected operating-system services, while file-system management covers file creation, deletion and access (Eduvos, 2026a, slides 41 and 43). In sequence, the application would open the customer's statement file, seek to the relevant period, read the data and close the file, with each step mediated by the corresponding system call above.

System call	Purpose	Importance for the bank statement
create / delete	Create a new file or remove an existing one.	Used when generating a new statement or applying an authorised retention rule; the OS prevents arbitrary file changes.
open / close	Open returns a protected descriptor or handle; close releases it and related kernel resources.	Gives the application controlled access to the correct statement and prevents leaked handles after viewing or printing.
read / write	Transfer bytes from a file to memory or from memory to a file/device.	Read obtains the statement data; write supports an authorised export or sends output through the printer spooler.
seek / reposition	Move the current file offset to a required position.	Lets the application retrieve a selected period or section without reading the complete file sequentially.
get / set attributes	Read or change metadata such as size, dates, ownership and permissions.	Supports validation and audit information; setting sensitive attributes remains permission-controlled.

Together, these calls enforce permissions, isolate one customer's process from another customer's data, coordinate simultaneous access, provide consistent error handling, support caching and buffering, and create an auditable access path. They also hide device-specific details, so the application can keep the same interface if the bank changes its disks or printers. Therefore, system calls turn a risky hardware operation into a controlled operating-system service (The Open Group, 2024).

1.2 Consequences of direct hard-drive and printer access

The developer's performance argument is understandable because avoiding a mode switch might appear to remove overhead. However, the small saving would introduce much larger risks. First, **security** would be weakened. Direct access could bypass normal permission checks, process isolation and protected file handles. A defect or malicious input could expose statements, transaction records or audit logs belonging to other customers. Modern operating systems deliberately separate user mode from kernel mode so that ordinary applications cannot freely control hardware (Microsoft, 2025a).

Second, **data integrity and reliability** would be threatened. Raw disk access can ignore file-system metadata, journaling, locks and cache-coherency rules. One incorrect block address could corrupt a statement, a transaction database or an entire volume. Direct printer control can also bypass the spooler, causing mixed jobs, lost pages or a blocked device. Third, **concurrency** becomes unsafe because thousands of banking sessions may issue conflicting operations without the OS scheduler, queues and locking mechanisms. Fourth, **maintenance and portability** become worse because the application would need device-specific driver logic for every disk controller and printer model. Finally, real performance may decline because the design gives up optimised OS features such as caching, buffering, direct memory access, I/O scheduling and printer spooling (Eduvos, 2026a; Microsoft, 2025b).

1.3.1 CIO decision

I would **reject the proposal**. A banking system should not trade its security boundary, auditability and data integrity for a speculative performance improvement.

1.3.2 Justification

System calls are the controlled gateway between an application running in user mode and privileged operating-system services. A call triggers a trap or software interrupt, switching the CPU mode bit from user mode to kernel mode. The interrupt vector and system-call number route the request through the system-call table to the correct kernel handler, which validates and executes it before returning a result or error code and restoring user mode. This matches the course distinction between user mode and kernel mode (Eduvos, 2026a, slide 41) and protects the application, hardware and other processes. In a regulated bank, confidentiality, correctness, recovery and audit trails outweigh minor call overhead. If profiling reveals an I/O bottleneck, the bank should use supported options such as asynchronous I/O, batching, caching, faster storage or an approved driver update rather than bypassing the OS.

Question 2 - Multithreaded ShopFast server

2.1a Why the single-threaded server performed poorly

The server handled requests serially, so one request had to finish before the next request could make progress. During Black Friday, requests arrived faster than that one thread could complete them. The waiting queue therefore grew, response time increased, requests timed out and customers abandoned their carts. The design also failed to use the quad-core machine properly because only one thread could run application work at a time.

2.1b Blocking and its effect on clients

Blocking happens when a thread cannot continue until an event completes. For example, a ShopFast request may wait for a product database query, payment gateway response, disk read or slow network client. In the single-threaded design, that blocked request stops the only server thread. A customer browsing products must then wait even though their request is unrelated to the blocked payment request. This reduces both responsiveness and throughput (Eduvos, 2026b).

2.2a Three benefits of multithreading

- **Economy:** creating and switching threads is cheaper than creating separate processes because less operating-system state and memory are required.
- **Resource sharing:** threads in the same process share the address space, code, data and open file descriptors, making cooperation faster and simpler.
- **Scalability:** runnable threads can execute in parallel across the four CPU cores (Eduvos, 2026b, slide 46).

2.2b Direct application to ShopFast

- **Economy:** during Black Friday, reusing lightweight request threads reduces creation and context-switching overhead, leaving more CPU time and memory for customer requests.
- **Resource sharing:** ShopFast's request threads can use the same configuration, product cache, connection pools and open resources without duplicating them for every customer.
- **Scalability:** product browsing, cart updates and payment requests can run on different cores at the same time, increasing peak throughput provided the database does not become the next bottleneck.

2.3a Task parallelism compared with data parallelism

Task parallelism assigns different tasks or functions to different processing units. The tasks may execute different code and work on different data. **Data parallelism** divides a dataset into parts and applies the same operation to each part at the same time. A useful example is calculating the same discount rule across four sections of a very large product list (Eduvos, 2026b, slide 47).

2.3b Classification of the ShopFast server

The redesigned server mainly exhibits **task parallelism**. First, each HTTP request is an independent task belonging to a particular customer. Second, requests can follow different code paths: one thread may browse products, another may update a cart and another may process a payment. These tasks can run concurrently across the four cores even though their operations and data are different. Some repeated handlers may resemble data parallelism, but the overall server design is best classified as task parallelism.

2.4a Thread pool

A thread pool is a managed set of reusable worker threads. The server creates or maintains workers independently of individual requests. When a worker finishes one request, it returns to the pool and can process another request.

2.4b Why pooling is preferred

Creating a new thread for every request adds creation and destruction overhead and can allow a traffic spike to create thousands of threads. That consumes memory, increases context switching and can crash the server. A bounded pool reuses threads and limits the amount of work running at once, giving ShopFast predictable resource use. This follows the course description of fixed reusable workers as a way to control overhead and resources (Eduvos, 2026b, slide 54). The best worker count must still be measured: CPU-heavy work may stay near the core count, while I/O-heavy work may benefit from more than four workers (Oracle, 2025).

2.4c Role of the task queue

The task queue sits between request acceptance and worker execution. Each accepted HTTP request becomes a task and waits in the queue until a worker is available. The next worker removes a task, processes it and returns to the pool. The queue absorbs short bursts and can enforce first-in-first-out

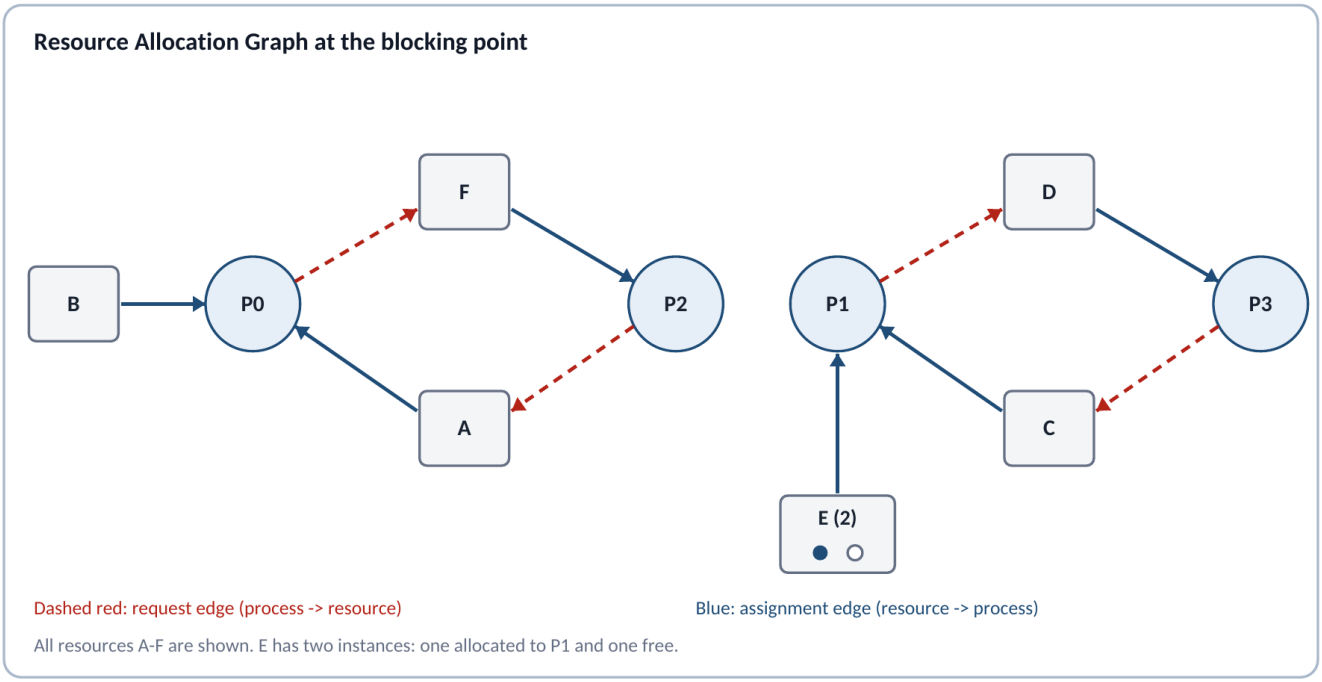
order or priorities. If it is bounded, it also provides backpressure: when both workers and queue are full, the server can reject or shed load in a controlled way instead of exhausting memory. ShopFast should monitor queue length and waiting time because a permanently growing queue means the system is overloaded, not healthy.

Question 3 - Deadlock in the manufacturing system

3.1 Resource Allocation Graph and deadlock result

Executing the resource requests line-by-line from P0 to P3 produces the following state. P0 obtains A, P1 obtains C, P2 obtains F and P3 obtains D. On the next line, P0 obtains B and P1 obtains one instance of E, while P2 waits for A and P3 waits for C. On the third request, P0 waits for F and P1 waits for D. The second instance of E is still free, but none of the blocked processes needs it next.

Request round	P0	P1	P2	P3
1	gets A	gets C	gets F	gets D
2	gets B	gets E1	waits for A	waits for C
3	waits for F	waits for D	already blocked	already blocked



The graph contains two cycles: P0 -> F -> P2 -> A -> P0 and P1 -> D -> P3 -> C -> P1. In the first cycle, P0 holds A and waits for F, while P2 holds F and waits for A. In the second cycle, P1 holds C and waits for D, while P3 holds D and waits for C. Since A, C, D and F each have one instance, the cycles confirm deadlock (Eduvos, 2026c, slide 66; Silberschatz, Galvin and Gagne, 2019). The unused second instance of E cannot break either cycle because no blocked process is waiting for E; P1 already holds E1 and is waiting for D.

3.2 Safe request order

I would impose one global resource order: **A < B < C < D < E < F**. Every process must request only in ascending order and should release in reverse order. The modified requests are:

Process	Original order	Safe order	Reason for change
P0	A -> B -> F	A -> B -> F	Already ascending
P1	C -> E -> D	C -> D -> E	D before E
P2	F -> A	A -> F	A before F
P3	D -> C	C -> D	C before D

This rule removes the circular-wait condition, one of the four necessary conditions for deadlock. A process holding a higher-ranked resource can never request a lower-ranked one, so a closed cycle of waiting edges cannot form. It also preserves the two instances of E; both are treated as the same resource class in the ordering (Eduvos, 2026c, slides 65 and 70; Silberschatz, Galvin and Gagne, 2019).

3.3 Prevention or avoidance

Deadlock prevention is more suitable for this manufacturing system. The four processes repeat known operations and use a small, fixed set of resources, so the company can enforce a global request order in the control software. The rule is simple, deterministic and easy to test, which matters in a physical production line where a deadlock can stop machinery and delay output. Deadlock avoidance, such as the Banker's algorithm, would check whether each allocation keeps the system in a safe state. It can allow more flexible allocations, but it needs accurate maximum-demand information and adds a decision step to every request. That overhead and complexity are unnecessary here because the resource pattern is predictable. The prevention approach may reduce some concurrency, but the safety and operational simplicity justify that trade-off (Eduvos, 2026c; Silberschatz, Galvin and Gagne, 2019).

Question 4 - File management and caching

4.1 Three advantages of directories

- **Organisation:** related files can be grouped by department or purpose, making employee records easier to locate and maintain.
- **Name separation:** different directories can contain files with the same name without a conflict because the full paths are different.
- **Security and administration:** permissions, backup rules and quotas can be applied to a directory and inherited by the files inside it. POSIX also describes files as a hierarchy in which directories form the non-terminal nodes (The Open Group, 2024).

4.2 Full path and Command Prompt line

The file is under Root (C:), then IT, then Software. Its full path is:

```
C:\IT\Software\Antivirus.exe
```

A Command Prompt line that runs it using the full path is:

```
"C:\IT\Software\Antivirus.exe"
```

The quotation marks are safe practice because they also work when a future directory name contains spaces. A fully qualified Windows path starts with the drive and follows each parent directory to the target file (Microsoft, 2025c).

4.3 HR and Public contents after the actions

HR directory	Public directory
Employees.xlsx Salaries.xlsx Contracts.docx Policies.pdf (copied in)	Policies.pdf Forms.docx

Public still contains Policies.pdf because the administrator **copied** the file rather than moving it. Deleting JanBackup.zip changes only IT\Backup. Renaming Expenses.xlsx changes only Finance, where the new name becomes Expenses_2026.xlsx. Therefore those two actions do not alter HR or Public.

4.4 Delete-on-logout compared with keep-until-deleted

A system that automatically deletes user files at logout or job termination has strong **security and storage-management** benefits. Temporary data does not remain for the next user on a shared computer, abandoned jobs do not fill the disk, and administrators spend less time cleaning old files. This approach suits kiosks, examination labs, temporary cloud jobs and systems that process sensitive short-lived data. Its main weakness is the risk of permanent data loss. A crash, forgotten save request or misunderstood policy can erase legitimate work. It also places a continuous burden on users to decide what must be preserved.

A system that keeps files until the user deletes them gives better **continuity and recovery**. Users can log out and continue later, records remain available for audits, and an accidental logout does not destroy work. This suits personal computers, office systems and business records. The disadvantages are that storage fills with stale files, private information remains exposed for longer, and users often fail to clean up or apply retention rules. Keeping everything can also increase backup cost and make searches less effective.

Neither extreme is ideal for every system. I would use a hybrid policy: automatically delete known temporary files and session caches; keep deliberately saved user documents; use quotas, retention periods and warnings; and provide a recycle or backup window for recovery. The policy should match the value, sensitivity and legal retention needs of the data.

4.5a Why systems treat file types differently

Some systems track file type because the operating system can then choose a suitable application, icon and permitted operation. For example, Windows file associations determine which application launches when a user opens a particular extension and which actions appear in the interface (Microsoft, 2021). File types can also help the OS distinguish regular files, directories, executables, devices and other special objects, improving validation and protection.

Other systems leave type information to the user or application, often through a filename extension or the file's internal format. This is simpler and flexible because the operating system can treat the content as a sequence of bytes. Systems that do not implement many file types reduce kernel complexity and avoid locking applications into a fixed type system. The trade-off is that applications must interpret the data correctly, and a misleading extension may cause the wrong program to be selected.

4.5b Which system is better?

Application-managed typing is better for a general-purpose system, provided the operating system retains a small set of structural file types. Applications can support new formats through filename extensions, registered associations, MIME information or content signatures without requiring kernel changes, which improves flexibility and portability. The operating system should still distinguish directories, regular files, symbolic links and devices because those types affect protection and device control. Strong OS-managed typing is preferable in tightly controlled or safety-critical environments, but for an ordinary desktop or server system, application-managed detailed formats combined with OS-managed structural types provide the best balance of flexibility and safety.

4.6a How caches improve performance

A cache keeps recently or frequently used data in a faster storage layer. When the requested item is already present, the system serves a **cache hit** instead of repeating a slower disk, network or main-memory access. This reduces average access time and traffic to the slower resource. Caches work because programs often show **temporal locality**, meaning recently used data is likely to be used again, and **spatial locality**, meaning nearby data is likely to be needed soon. The course slides describe cache management as temporarily storing frequently accessed data in faster storage (Eduvos, 2026a, slide 43). File-system caches can therefore avoid repeated disk reads, while CPU caches reduce delays caused by main memory.

The improvement can be expressed as **average access time = hit time + (miss rate x miss penalty)**. For example, if a cache hit takes 1 ms, the miss rate is 10%, and a miss adds another 9 ms, the average is 1 + (0.10 x 9) = **1.9 ms**, compared with approximately 10 ms when every request uses the slower layer. Coherency and invalidation rules are still required so that faster access does not return stale data (Linux Kernel, 2026).

4.6b Why systems do not use unlimited or larger caches

Fast cache memory is limited, expensive and consumes chip area or system RAM. A larger cache can improve the hit rate, but the benefit eventually becomes smaller because not all workloads reuse the extra data. Larger structures may also take longer to search, use more power, require more metadata and make coherency between cores or devices more complex. At file-system level, a very large cache can take memory away from applications and must still manage dirty data, invalidation and recovery. It can also retain useless data and cause cache pollution. The best cache size is therefore workload-specific: Intel

notes that cache effectiveness depends on whether the program's critical code and data fit the cache, and a poorly matched cache may provide little or even negative benefit (Intel, 2023).

Reference list

Eduvos. 2026a. *ITOPA3-T11 Week 0 and 1 Lessons 1-4*. Internal course slides, especially slides 41 and 43.

Eduvos. 2026b. *ITOPA3-T11 Week 2 Lessons 5-7*. Internal course slides, especially slides 46, 47 and 54.

Eduvos. 2026c. *ITOPA3-T11 Week 3 Lessons 8-10*. Internal course slides, especially slides 65, 66 and 70.

Intel. 2023. *Effective Use of Cache Memory*. Nios II Processor Reference Guide. Available at: https://www.intel.com/content/www/us/en/docs/programmable/683836/current/effective-use-of-cache-memory.html (Accessed 27 August 2026).

Linux Kernel. 2026. *Network Filesystem Caching API*. Available at: https://docs.kernel.org/filesystems/caching/netfs-api.html (Accessed 27 August 2026).

Microsoft. 2021. *How File Associations Work*. Available at: https://learn.microsoft.com/en-us/windows/win32/shell/fa-how-work (Accessed 27 August 2026).

Microsoft. 2025a. *User Mode and Kernel Mode*. Available at: https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode (Accessed 27 August 2026).

Microsoft. 2025b. *Windows Security Model for Driver Developers*. Available at: https://learn.microsoft.com/en-us/windows-hardware/drivers/driversecurity/windows-security-model (Accessed 27 August 2026).

Microsoft. 2025c. *About Path Syntax*. Available at: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_path_syntax (Accessed 27 August 2026).

Oracle. 2025. *ThreadPoolExecutor*, *Java SE 24 API*. Available at: https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/concurrent/ThreadPoolExecutor.html (Accessed 27 August 2026).

Silberschatz, A., Galvin, P.B. and Gagne, G. 2019. Silberschatz's Operating System Concepts, Global Edition. 10th ed. Wiley. ISBN 9781119454083. Available at: https://bcs.wiley.com/he-bcs/Books?action=index&bcsId=11582&itemId=1119454085 (Accessed 28 August 2026).

The Open Group. 2024. *POSIX.1-2024 General Concepts and System Interfaces*. Available at: https://pubs.opengroup.org/onlinepubs/9799919799/ (Accessed 27 August 2026).